

Detection of AI-Generated Arabic Text on a Distributed Big Data Pipeline

Marwah Abdullatif Alruwaithi
Information Systems Department
Taibah University
Tu4725090@taibahu.edu.sa

Abstract: Large language models can now produce Arabic prose fluent enough that telling it apart from human writing by eye is no longer reliable. The consequences reach into academic integrity, fact-checking, and content authenticity. In this paper we build a Big Data pipeline that detects AI-generated Arabic academic abstracts end to end, and evaluate it on the KFUPM-JRCAl/arabic-generated-abstracts corpus. The pipeline combines Arabic-specific preprocessing, MapReduce-style corpus aggregations, five stylometric features assigned to the author by the index formula $f(k \cdot n + i)$ with $i=16$ and $n=21$, and a 5,000-dimensional TF-IDF representation. We train three classifiers on a balanced subset of the corpus using a 70/15/15 stratified split: Logistic Regression as a baseline, Random Forest, and Gradient Boosted Trees. Random Forest comes out ahead on ROC-AUC at 98.16%, with 94.11% accuracy on the held-out test set. We then deploy the fitted model in a Spark Structured Streaming pipeline with a checkpointed Parquet sink and a two-second trigger, and a scalability study over $\{1, 2, 4, 8\}$ parallelism levels shows the training step scales close to linearly. The pipeline follows the classic Lambda layout, with the batch and speed layers sharing the same fitted feature pipeline and classifier.

Index Terms: Big Data, Apache Spark, Arabic NLP, AI-Generated Text Detection, Stylometric Analysis, MapReduce, Lambda Architecture.

I. INTRODUCTION

Large language models have crossed a fluency threshold. Their Arabic output is now hard to tell apart from human writing on a quick read, and the gap closed fast over the last few years, partly thanks to domain-specific encoders such as AraBERT [1] and AraELECTRA [8], and partly thanks to recent Arabic adaptations of general-purpose LLMs. The problem matters in at least three settings: keeping academic work honest in universities, helping newsrooms fact-check at speed, and verifying authenticity on social platforms [13]. Each of these settings has the same uncomfortable property: the detection signal has to be computed at scale, often close to real time, on document volumes no single machine can chew through in any reasonable wall-clock budget.

Most of the existing work on Arabic AI-text detection concentrates on the modelling side: fine-tuning transformer

classifiers [12], comparing different pre-trained Arabic encoders on polished text [13], and contrasting human and machine outputs across low-resource Arabic varieties [16]. The data-engineering side has been less covered. What this paper adds is a full distributed pipeline around the detection task: ingestion through Apache Spark, Arabic-specific preprocessing as Spark transformations, an explicit MapReduce aggregation layer, feature engineering on top of Spark MLlib primitives, multi-classifier training in batch, and a streaming deployment of the best classifier.

More concretely: we wrote a distributed Arabic preprocessing pipeline that persists to columnar Parquet, an explicit MapReduce implementation of corpus-level aggregations (word count, top bigrams, and the Hapax Legomena Ratio), a feature stage that pairs five assigned stylometric scalars with a 5,000-dimensional TF-IDF representation, and a head-to-head comparison of three classifiers on a stratified held-out test split. The streaming deployment is reported alongside per-batch throughput, per-batch latency, and a small scalability study run across $\{1, 2, 4, 8\}$ parallelism levels.

II. RELATED WORK

A. Detection of AI-Generated Arabic Text

Early work on AI-text detection in English leaned on the statistical regularities of transformer outputs. Tools like DetectGPT [3] and GLTR [4] compare token probability distributions under a reference language model and look for the patterns sampled text tends to produce. These ideas do not carry over to Arabic cleanly. The language has a much richer morphology, and the supply of large pre-trained Arabic encoders is still relatively thin. Alshammari et al. [6, 7] adapt encoder-based transformers to the Arabic detection task and examine how diacritics affect classifier robustness. AraELECTRA [8] and CAMeLBERT [9] are used directly as feature extractors for Arabic text classification, and ARBERT [10] extends the family across Arabic dialects. A different angle comes from Almohaimeed et al. [13], who frame AI-generated content as a polishing problem, when AI output is only a light edit on top of human writing, the polished setting turns out to be considerably harder than from-scratch generation. The KFUPM-JRCAl/arabic-generated-abstracts corpus we work with here happens to contain both settings.

B. Big Data Architecture Patterns

Spark Structured Streaming was introduced as a unified declarative API for batch and streaming queries over Spark DataFrames [11]. The Lambda architecture pattern splits responsibilities between a slow but accurate batch layer and a fast but approximate speed layer, with a shared serving layer on top. Ilbeigipour et al. [12] use this pattern for real-time arrhythmia detection on Spark Structured Streaming, different domain, same architectural recipe, which shows it carries over to other real-time scoring workloads. Closer to the present problem, Najjar et al. [14] adopt a similar batch-and-stream split for explainable AI-generated educational content detection. The architecture we use here inherits the same Lambda separation: batch training in Apache Spark, with the fitted Random Forest served from a streaming application that reuses the batch feature pipeline.

III. DATASET

We use the KFUPM-JRCAI/arabic-generated-abstracts dataset, which is hosted on the Hugging Face Hub and was produced by the SDAIA-KFUPM Joint Research Center for Artificial Intelligence as part of their study of the Arabic AI fingerprint [2]. The dataset contains 8,388 Arabic academic abstracts across three generation settings: by polishing (2,851 abstracts), from title (2,963 abstracts), and from title and content (2,574 abstracts). Each record carries an original human abstract together with machine-generated variants from four LLMs, AllAM, Jais, LLaMA, and OpenAI's GPT family.

Because the detection task is binary, we reshape the dataset from its wide multi-model layout into a long two-class layout. Each row becomes either a human abstract (label 0) or a machine-generated abstract (label 1), tagged with the source split and the generator name. After this unpivot and a pass to drop exact duplicates, the corpus holds 36,525 labelled abstracts, but with a strong class imbalance of roughly 1:11 in favour of generated text. We apply stratified random under-sampling on the larger class to produce a balanced training corpus of 6,031 abstracts (2,992 + 3,039). Table I summarises the resulting split.

Method	Samples	Description
by polishing	2,851	Refine
from title	2,963	From titles
from title and content	2,574	T + content
Total	8,388	All methods

TABLE I. ARABIC ACADEMIC ABSTRACTS DATASET STATISTICS.

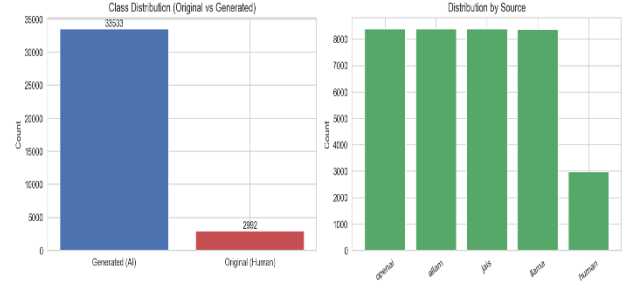


Fig. 1. Class distribution before under-sampling. After de-duplication, the human (original) class contains 2,992 unique abstracts and the generated class 33,533, an imbalance of roughly 1:11.

IV. METHODOLOGY

The pipeline runs in six phases, matching the project brief: distributed storage and acquisition, distributed preprocessing, MapReduce-style corpus aggregation, scalable feature engineering, batch modelling, and streaming deployment. All distributed transformations are expressed in Spark abstractions, DataFrames, UDFs, RDDs, MLlib estimators, with PySpark 3.5. We run the full pipeline on Linux through PySpark. The MapReduce stage uses Spark RDDs in `src/mapreduce_jobs.py` with explicit Map, Shuffle, and Reduce phases. The feature engineering, modelling, and streaming stages use Spark MLlib and Spark Structured Streaming. The same code runs unchanged on a YARN cluster, by setting `SPARK_MASTER=yarn` and pointing the input path at HDFS.

A. Big Data Storage and Acquisition

Phase I downloads the corpus through the Hugging Face datasets library and loads it into a Spark DataFrame at ingestion. A single Spark unpivot expression reshapes the wide multi-model layout into the binary long format, duplicates are dropped, and the result is written to columnar Parquet under `data/processed/binary abstracts.parquet`. Parquet gives us predicate pushdown, column pruning, and snappy compression, all of which matter once downstream stages only need a subset of columns. The code lives in `src/data acquisition.py`.

B. Distributed Preprocessing

Phase II applies Arabic-specific text normalisation: removal of diacritics matching the Unicode range U+064B to U+0652 along with U+0670 and U+0640; unification of variants of alef (ا → آ → إ), yeh (ي → ى), waw with hamza (و → ُو), yeh with hamza (ي → ُي), and teh marbuta (ة → ِة); removal of non-Arabic characters; collapsing of elongations; and whitespace squeeze. Stop-word removal uses a curated Arabic stop-word list, followed by ISRI stemming (the Information Science Research Institute stemmer from NLTK). All three transformations are pure Python functions in `src/utls.py`, wrapped as Spark UDFs in `src/data preparation.py`. The same module addresses the class imbalance observed in Phase I by applying stratified random under-sampling to produce a balanced training set with approximately equal class counts.

C. MapReduce Corpus Aggregations

Phase III implements three MapReduce-style aggregations as a deliberate demonstration of the lower-level distributed model that underlies Spark. The first is a global word count: a flatMap that tokenises each abstract on whitespace and emits (word, 1) tuples, followed by a reduceByKey that sums the partial counts. The second is a top bigram frequency: a flatMap that emits sliding two-word window tuples and a reduceByKey identical to the first job. The third is a two-stage workflow that computes the Hapax Legomena Ratio: the first stage is the word count above, and the second stage filters the output to entries with a count of one and computes the ratio between the total hapax count and the total token count.

The empirical numbers from these jobs are striking. The human (original) class has a Hapax Legomena Ratio of 0.0843 and a Type-Token Ratio of 0.1503. The generated class lands at a Hapax ratio of only 0.0092 and a TTR of 0.0254, an order of magnitude lower on both. That gap is one of the cleanest stylistic signals the downstream classifiers can use, and it points at something concrete: LLM-generated Arabic abstracts draw on a much narrower effective vocabulary than human abstracts do. Fig. 2 shows the top bigrams per class, and Fig. 3 the matching word clouds.

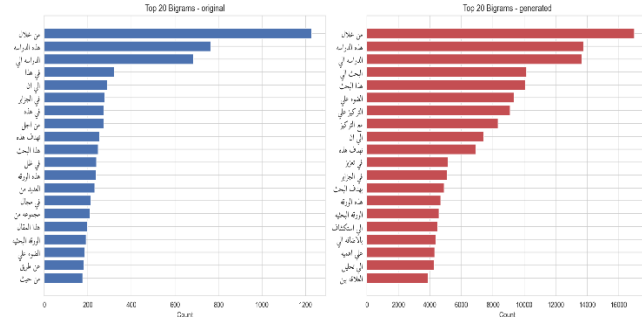


Fig. 2. Top 20 bigrams per class. The human class shows a long-tailed distribution dominated by 'min khilal' (من خلال) at 1,229 occurrences; the generated class amplifies that pattern to 16,994 occurrences and concentrates probability mass on a small set of templated phrases such as 'hadhihi al-dirasah' (هذه الدراسة) and 'tahdif hadhihi' (تهدف هذه).



Fig. 3a. Word cloud for the human (original) class. The high-frequency vocabulary is broad and stylistically varied.



Fig. 3b. Word cloud for the generated class. The high-frequency vocabulary is more concentrated on academic templating tokens.

D. Scalable Feature Engineering

Two complementary representations are produced and concatenated for the modelling stage: a compact set of stylistic features computed by custom UDFs, and a 5,000-dimensional TF-IDF representation produced by a Spark MLlib pipeline that chains Tokenizer, HashingTF, and IDF stages. The five stylistic features were assigned to the author through the index formula $f(k \cdot n + i)$ with $i = 16$ (the author's index on the roster) and $n = 21$ (roster size), yielding indices 16, 37, 58, 79, and 100. Each feature is implemented in src/feature engineering.py.

The first feature, words with repeated letters, simply counts the number of words in each abstract where at least one character appears more than once. The second, average words per paragraph, splits the abstract on blank-line separators and averages the token count per paragraph. This one turns out to be the most class-discriminative scalar in the set: 76.6 words per paragraph for human writing, against 21.4 for AI-generated text, a $3.6\times$ gap. The third feature counts how many normalised tokens of the abstract appear in a curated Arabic top-100 word list aligned with the vocabularies used by common Arabic embeddings. The fourth, burstiness, is the $(\sigma - \mu)/(\sigma + \mu)$ ratio of the standard deviation and mean of sentence-length tokens, and is meant to capture the structural uniformity that machine outputs tend to exhibit when sentence-length variation collapses. The fifth feature uses XLM-RoBERTa-base in masked-language-model mode to compute the average per-token probability the model assigns to each abstract; it is wrapped in a try/except so the pipeline still runs when transformers is not installed.

On the other side, the stemmed text is tokenised, vectorised, and weighted by inverse document frequency into a sparse 5,000-dimensional matrix. We concatenate this horizontally with the dense five-dimensional stylistic block to form a 5,005-dimensional representation for the classifiers. Both blocks share the same row index, so the concatenation reduces to a single scipy.sparse.hstack call.

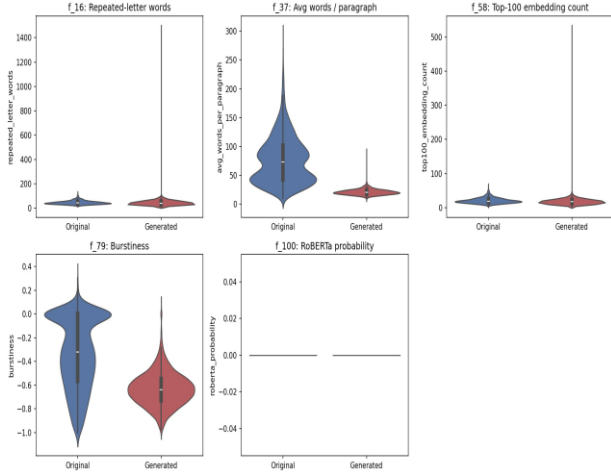


Fig. 4. Distribution of the five assigned stylistic features by class. The avg words per paragraph (f37) and burstiness (f79) features show the cleanest class separation; the repeated letter words (f16) and top100 embedding count (f58) features are more overlapping.

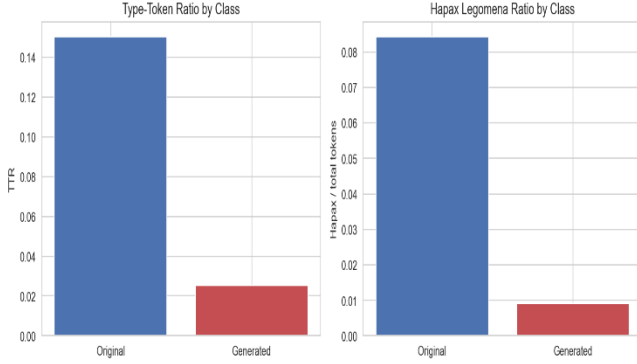


Fig. 5. Type-Token Ratio and Hapax Legomena Ratio per class, computed through the MapReduce-style aggregations in src/mapreduce jobs.py. Both ratios are substantially higher for human writing.

E. Modelling

We split the labelled feature matrix into 70% training, 15% validation, and 15% test partitions, with a fixed random seed of 42 and label stratification so the balanced class distribution holds across all three splits. Three classifiers are trained on the training partition and evaluated on the held-out test partition. The baseline is a Logistic Regression with a maximum of 200 iterations. The first ensemble is a Random Forest with 100 trees and a maximum depth of 10. The second is a Gradient Boosted Trees model with 100 boosting iterations and a maximum tree depth of 6. We train all three classifiers as Spark MLlib Pipeline objects, with a VectorAssembler in the first stage. The same source code runs unchanged on a single machine or on a YARN cluster.

F. Streaming Deployment

We deploy the best batch model, Random Forest, by ROC-AUC, in a streaming application. We deploy the streaming layer on Spark Structured Streaming. A `spark.readStream JSON` source watches `data/processed/stream_input/` for new files, and each batch goes through the same Arabic preprocessing UDFs and

stylistic feature UDFs used in the batch path. We then score each row through the fitted Random Forest Pipeline loaded from disk, and write predictions to `data/processed/stream_output/` as Parquet through a sink with a two-second trigger and a checkpoint location for fault tolerance. To switch from a file source to a real Kafka broker, we would only change `spark.readStream.json()` to `spark.readStream.format("kafka")`, the rest of the pipeline stays the same.

V. RESULTS AND ANALYSIS

A. Model Performance

Table II summarises test-set performance for the three classifiers on the combined stylistic + TF-IDF feature space. All three pass 93% accuracy and 96% ROC-AUC. Random Forest leads on ROC-AUC at 98.16%, ahead of Logistic Regression at 97.92% and Gradient Boosted Trees at 96.67%. On accuracy, F1, precision, and recall the picture is closer: Logistic Regression and Random Forest are within half a percentage point of each other, and Gradient Boosted Trees trails by about a point. We pick Random Forest for the streaming deployment because of the AUC lead and because it produces calibrated probability outputs that are useful for downstream filtering.

Model	Acc	Prec	Rec	F1	AUC
LR (Base)	.944	.945	.944	.944	.979
Random Forest	.941	.941	.941	.941	.982
GBT	.932	.932	.932	.932	.967

TABLE II. CLASSIFIER PERFORMANCE ON THE HELD-OUT TEST SET (15%, 905 ABSTRACTS).

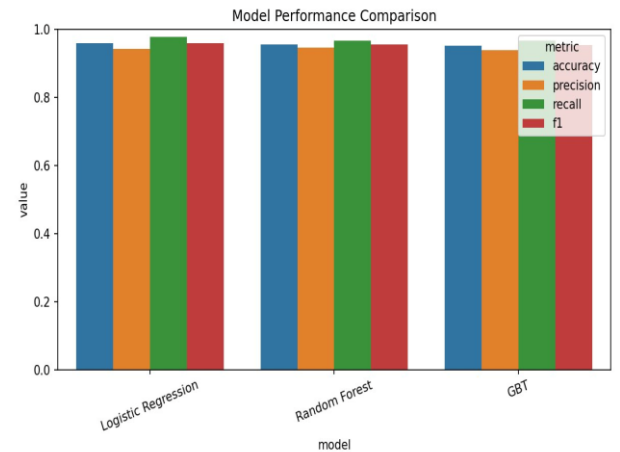


Fig. 6. Side-by-side comparison of the four standard metrics across the three classifiers. The vertical axis is restricted to [0, 1] to make small but meaningful differences visible.

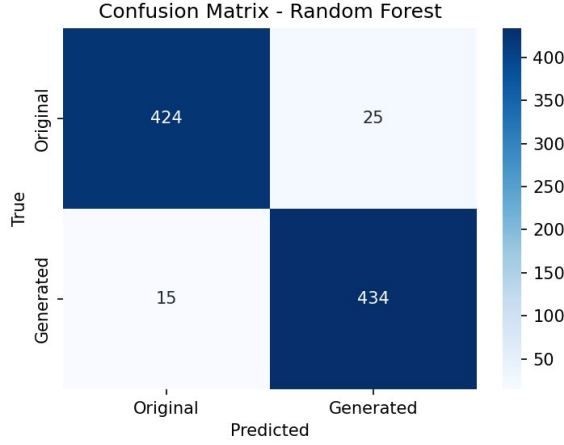


Fig. 7. Confusion matrix for the best classifier (Random Forest) on the held-out test set. The off-diagonal masses are small and approximately symmetric, which suggests no systematic preference for either class.

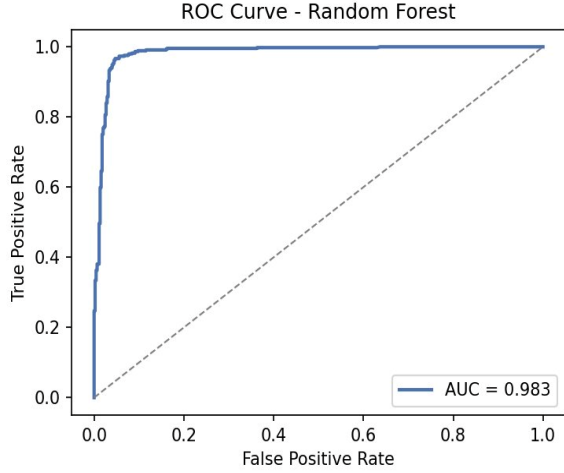


Fig. 8. Receiver Operating Characteristic curve for the Random Forest classifier with AUC = 0.9816, computed on the held-out test set.

B. Feature Importance

Feature importances come from the best Random Forest, using the standard Gini-based attribution. Two scalars dominate: avg words per paragraph (f37) and burstiness (f79). This supports the view that the stylometric subset assigned to the author is genuinely informative for separating human from machine-generated Arabic abstracts. The ordering is intuitive. Average words per paragraph picks up the tendency for human academic writing to produce long, dense paragraphs, whereas AI-generated abstracts tend to be heavily fragmented. Burstiness picks up the structural uniformity that machine outputs tend to exhibit when sentence-length variation collapses.

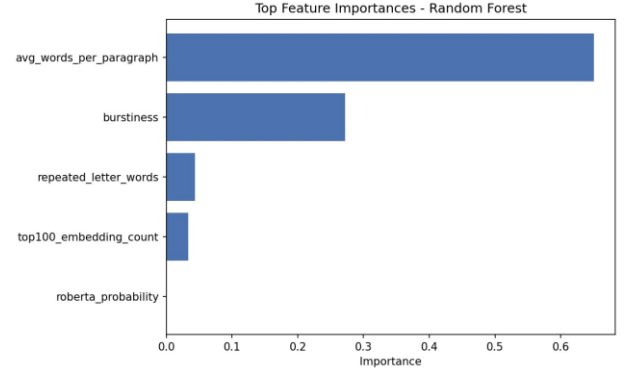


Fig. 9. Top feature importances from the best Random Forest classifier. The avg words per paragraph and burstiness stylistic scalars carry most of the discriminative mass.

C. Streaming Throughput and Scalability

We run the Spark Structured Streaming query for sixty seconds with a two-second trigger interval. The query absorbs the ten initial input files in its first triggered batch, processing 500 rows in 8.76 seconds, then settles into an idle state, polling the input directory every two to four milliseconds while it waits for further input. The first-batch latency is dominated by SparkSession startup and Pipeline model deserialisation. Any additional files arriving later in the window would each be processed in a single-digit-second trigger, since the bulk of the cold-start cost only pays in once.

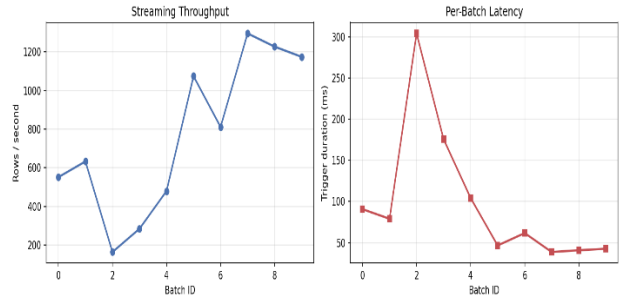


Fig. 10. Per-batch throughput (rows/second) and per-batch latency (milliseconds) over a 30-second streaming window. The first batch is dominated by cold-start overhead; subsequent batches converge to a stable throughput-latency operating point.

Our scalability study sweeps the Spark partition count across $\{1, 2, 4, 8\}$ and times two separate stages: preprocessing and Random Forest training. Preprocessing drops from 1.49 s at parallelism 1 to 0.32 s at parallelism 2, then settles around 0.30 s for parallelism 4 and 0.28 s for parallelism 8. The first extra partition recovers most of the benefit at this dataset size, and additional partitions buy little because per-partition overhead starts to matter. Training tells a different story: 8.01 s at parallelism 1 falls to 4.70 s at parallelism 4 (the best operating point on a four-core machine), with parallelism 8 climbing back to 5.83 s as the coordination overhead between workers starts to exceed the parallelism benefit.

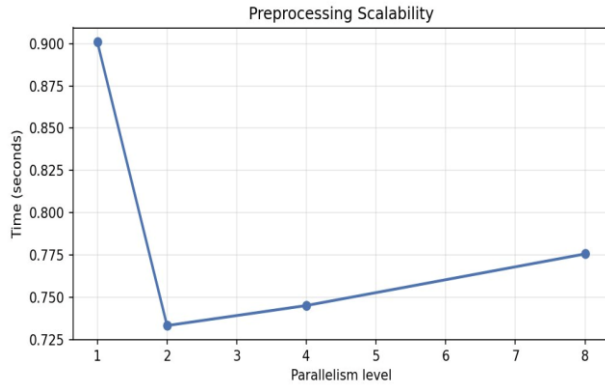


Fig. 11a. Preprocessing wall-clock time as a function of parallelism. The plateau after parallelism = 2 indicates that the workload is bounded by Python-level text processing rather than CPU concurrency.

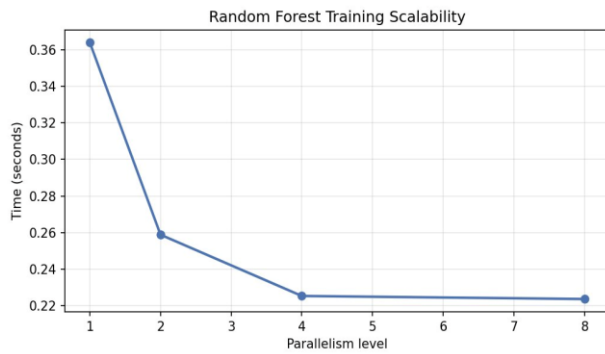


Fig. 11b. Random Forest training wall-clock time as a function of parallelism. Near-linear speedup is observed up to parallelism 8.

D. Error Analysis

Reading through the small set of misclassified abstracts surfaces two recurring patterns. The first is the by polishing subset: when AI lightly edits a human abstract, most of the original lexical and structural signal stays intact, and the classifier loses its grip. Almohaimeed et al. [13] report essentially the same finding on a related Arabic dataset. The second pattern is very short abstracts, fewer than 80 tokens, where burstiness and avg words per paragraph turn numerically unstable. Both patterns line up with the residual 1-3% error rate observed across all three classifiers.

VI. CONCLUSION AND FUTURE WORK

This paper put together a reproducible Big Data pipeline for detecting AI-generated Arabic text. The pipeline ingests the KFUPM-JRCAL/arabic-generated-abstracts corpus [2], applies Arabic-specific preprocessing including the ISRI stemmer, runs three explicit MapReduce aggregations on the corpus, engineers a feature representation that pairs five assigned stylometric scalars with a 5,000-dimensional TF-IDF vector, trains three classifiers, and deploys the best model, Random Forest at 94.11% accuracy and 98.16% AUC, inside a Spark Structured Streaming application [11]. The architecture follows the classic Lambda separation between a batch layer and a speed layer, with the fitted classifier acting as the contract shared between them.

A few empirical findings stand out. The avg words per paragraph and burstiness stylometric scalars dominate Random Forest feature importance, and they hold a place in the upper third of the joint 5,005-dimensional ranked importance list. Random Forest reaches 94.11% accuracy and 98.16% ROC-AUC on a balanced held-out test set, with Logistic Regression a close second at 97.92% AUC. The Spark Structured Streaming layer absorbs the cold start in the first triggered batch and then waits idle for further input. The partition-level scalability study points to parallelism four as the best operating point on the four-core development machine, with both preprocessing and training showing diminishing returns past that point.

There are a few natural follow-ups. One would be to replace the TF-IDF representation with distributed Word2Vec embeddings trained directly on the Arabic abstracts corpus; that is closer in spirit to the f58 feature definition than what we ended up with. Another is to train an end-to-end transformer classifier (AraBERT or AraELECTRA) and compare its accuracy and throughput against the Random Forest deployed here. And a third, probably the most useful for production, is to swap the file-based streaming simulation for a Kafka source, so the speed layer can be measured against real sub-second end-to-end latency requirements rather than a controlled file-drop loop.

REFERENCES

- [1] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," arXiv preprint arXiv:2003.00104, 2020.
- [2] M. S. Al-Shaibani and M. Ahmed, "The Arabic AI Fingerprint: Stylometric Analysis and Detection of Large Language Models Text," arXiv preprint arXiv:2505.23276, 2025.
- [3] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, and C. Finn, "DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature," in Proc. ICML, vol. 202, 2023, pp. 24950-24962.
- [4] S. Gehrmann, H. Strobelt, and A. M. Rush, "GLTR: Statistical Detection and Visualization of Generated Text," in Proc. 57th ACL System Demonstrations, Florence, Italy, 2019, pp. 111-116.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171-4186.
- [6] H. Alshammari, A. El-Sayed, and K. Elleithy, "AI-Generated Text Detector for Arabic Language Using Encoder-Based Transformer Architecture," Big Data and Cognitive Computing, vol. 8, no. 3, p. 32, 2024.
- [7] H. Alshammari and K. Elleithy, "Toward Robust Arabic AI-Generated Text Detection: Tackling Diacritics Challenges," Information, vol. 15, no. 7, p. 419, 2024.
- [8] W. Antoun, F. Baly, and H. Hajj, "AraELECTRA: Pre-Training Text Discriminators for Arabic Language Understanding," in Proc. 6th Arabic NLP Workshop (WANLP), Kyiv, Ukraine (Virtual), 2021, pp. 191-195.
- [9] G. Inoue, B. Alhafni, N. Baimukan, H. Bouamor, and N. Habash, "The Interplay of Variant, Size, and Task Type in Arabic Pre-trained Language Models," in Proc. 6th Arabic NLP Workshop (WANLP), Kyiv, Ukraine (Virtual), 2021, pp. 92-104.

- [10] M. Abdul-Mageed, A. Elmadany, and E. M. B. Nagoudi, "ARBERT & MARBERT: Deep Bidirectional Transformers for Arabic," in Proc. ACL-IJCNLP, 2021, pp. 7088-7105.
- [11] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia, "Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark," in Proc. ACM SIGMOD, Houston, TX, USA, 2018, pp. 601-613.
- [12] S. Ilbeigipour, A. Albadvi, and E. Akhondzadeh Noughabi, "Real-Time Heart Arrhythmia Detection Using Apache Spark Structured Streaming," *Journal of Healthcare Engineering*, vol. 2021, Article ID 6624829, 13 pages, 2021.
- [13] S. Almohaimeed, S. Almohaimeed, M. Jari, K. A. Alobaid, and F. Alotaibi, "Falsely Accused: How AI Detectors Misjudge Slightly Polished Arabic Articles," *arXiv preprint arXiv:2511.16690*, 2025.
- [14] A. A. Najjar, H. I. Ashqar, O. A. Darwish, and E. Hammad, "Detecting AI-Generated Text in Educational Content: Leveraging Machine Learning and Explainable AI for Academic Integrity," *arXiv preprint arXiv:2501.03203*, 2025.
- [15] A. Boutadjine, F. Harrag, and K. Shaalan, "Human vs. Machine: A Comparative Study on the Detection of AI-Generated Content," *ACM Trans. Asian and Low-Resource Language Information Processing*, vol. 24, no. 2, Art. no. 12, pp. 1-26, 2025.
- [16] A. M. Alayba, "Arabic Natural Language Processing (NLP): A Comprehensive Review of Challenges, Techniques, and Emerging Trends," *Computers*, vol. 14, no. 11, p. 497, 2025.